

Information Retrieval

Assignment 1 - Report

Ori Moscovitz

1 Introduction

The goal of this assignment was to implement an experimental retrieval system based on the vector space model and inverted index for English and Czech texts, with the aim of optimizing Mean Average Precision (MAP) and Precision at 10. This report details the system architecture and the experiments that were conducted and discusses the results for each language.

2 System overview & Building Instructions

The system is implemented entirely in Python. The system does **not** rely on any IR-specific frameworks and instead utilizes basic data structures and libraries such as nltk and ufal.morphodita for linguistic preprocessing.

The the results for the baseline (run-0) and the final approach result (run-1) can be generated via the cmd. For example:

```
python run.py -q topics-train_en.xml -d documents_en.lst -r 0 -o run-1_train_en.res // run-0
```

```
python run.py -q topics-train_en.xml -d documents_en.lst -r 1 -o run-1_train_en.res // run-1
```

where:

-q: path to the queries xml file

-d: path to a list of documents to be read

-r: which run to apply (0 or 1)

-o: name of the output file

Note: there is a validation for the queries path and the document path, to validate it reads either en and en or cs and cs. A mix of both will raise an error.

For further details please refer to the assignment [repository in Github](#).

The system consists of the modules:

Module	Responsibility
Run.py	Cmd interface entry point
Utils.py	Document loading, vectorization, query processing and result writing
File_handler.py	Document parsing, tokenization, stop-words removal, lemmatization, stemming, equivalence classes
Eval_handler.py	TF-IDf/BM25 vector computation, cosine similarity, ranking and PRF
Xml_handler.py	XML cleaning and validation
Plot_handler.py	Evaluation with trec_eval and plotting

3 Experiments

The experiments were conducted incrementally, building upon previous improvements. Each step's results were evaluated against the training qrels.

3.1 English Experiments:

3.1.1 Step-0: Baseline (run-0)

For the baseline the configuration is:

- Whitespace tokenization
- No case normalization
- No stop-word removal
- Natural term frequency
- No document frequency weighting
- Cosine normalization and similarity

Results: MAP = 0.0413 and P@10 = 0.076.

The baseline performs poorly, as we could expect. Raw term frequencies without normalization gives too much weight to common words, and the lack of the preprocessing means morphological variants are treated as isolated, distinct terms (e.g. "run", "runs" won't match) .

3.1.2 Step-1: Improved tokenization

Split on spaces, commas, punctuation, parentheses, and hyphens using regex.

```
tokens = re.split(r"[^\w]+", text.casefold())
```

Results: MAP = 0.1288 (+212%), P@10 = 0.172 (+126%)

The overwhelming improvement aligns with what we learned about tokenization challenges. Handling punctuation properly and splitting on hyphens captures more meaningful terms, such as "state-of-the-art", becoming three terms). The `casefold()` normalization also helps in treating terms such as "Dog" and "dog" as identical.

3.1.3 Step-2: Stop word removal

I have removed common English stop words using NLTK's stop words list.

```
def remove_stop_words(tokens, language):  
    stop_words = stopwords_en if language == "en" else stopwords_cs  
    return [t for t in tokens if t not in stop_words]
```

Results: MAP = 0.2111 (+64%), P@10 = 0.244 (+42%)

Stop word removal improves MAP by removing uninformative terms such as "a", "the", "an", etc. As we discussed in class, stop words appear in nearly every document and contribute little to differentiation between relevant and non-relevant documents.

3.1.4 Step-3: Remove empty tokens

Filtering out empty strings resulting from the tokenization approach.

```
tokens = [t for t in tokens if t]
```

Results: MAP = 0.2187 (+3.6%), P@10 = 0.272 (+11.5%)

3.1.5 Step-4: Lemmatization

I applied WordNet lemmatizer to reduce the dictionary size.

```
def lemmatize_en(tokens):  
    lemmatizer = WordNetLemmatizer()  
    return [lemmatizer.lemmatize(t) for t in tokens]
```

Results: MAP = 0.2253 (+3.0%), P@10 = 0.272 (no change)

Lemmatization provides a small improvement. Lemmatization merges inflectional variations ("running" → "run", "ran" → "run"). English morphology is fairly simple, so the gain is minor.

3.1.6 Step-5: Stemming

Applied Porter Stemmer (SnowballStemmer for English).

```
def stem_tokens(tokens, language):  
    if language == "en":  
        stemmer = SnowballStemmer("english")  
        return [stemmer.stem(t) for t in tokens]  
    return tokens
```

Results: MAP = 0.1958 (-13.1%), P@10 = 0.268 (-1.5%)

The stemming hurt the performance. The stemmer seemed to be over stemming, merging terms incorrectly. For example, it conflated "operating" and "operative" to the stem "oper", but they have different meanings in queries such as "operating system" vs. "operative dentistry". This aggressive stemming seems to hurt the precision without a recall gain for English.

3.1.7 Step-6: Equivalence classes

I have normalized abbreviations, numbers and repeated characters.

```
mapping = {  
    "u.s.": "usa", "u.k.": "uk", "britain": "uk",  
    "govt": "government", "corp": "corporation",  
    # ... and number normalization: \d+([\.,]\d+)? → <NUM>  
}
```

Results: MAP = 0.1906 (-2.7%), P@10 = 0.264 (-1.5%)

Equivalence classes also hurt the performance for English. One reason that may be is that the mapping may be incomplete. Additionally, some of the normalizations might be conflating distinct terms (e.g. "good" and "god" which should **not** be conflated). This suggests that for English, normalization requires careful tuning.

3.1.8 Step-7: TF-IDF Weighting

Replaced term frequency counters with TF-IDF vectors.

```
def compute_tfidf_vectors(docs_dict):
    N = len(docs_dict)

    tf_dict = {doc_id: Counter(tokens) for doc_id, tokens in
docs_dict.items()}
    df = Counter()
    for tokens in docs_dict.values():
        for term in set(tokens):
            df[term] += 1

    tfidf_dict = {}
    for doc_id, tf in tf_dict.items():
        vec = {}
        doc_len = sum(tf.values())
        for term, count in tf.items():
            tf_val = count / doc_len
            idf_val = math.log((N + 1) / (df[term] + 1))
            vec[term] = tf_val * idf_val
        tfidf_dict[doc_id] = vec

    return tfidf_dict
```

Results: MAP = 0.2356 (+23.6%), P@10 = 0.336 (+27%)

TF-IDF weighting drastically improves both MAP and P@10, as expected. As we discussed in class, the IDF weighting gives higher weight to rare, **informative** terms while decreasing the weight of common terms.

3.1.9 Step-8: Best Match 25 (BM-25)

Replaced the TF-IDF with Best Match 25.

```
def compute_bm25_vectors(docs_dict, k1=1.2, b=0.75):
    N = len(docs_dict)

    tf_dict = {doc_id: Counter(tokens) for doc_id, tokens in
docs_dict.items()}

    doc_lengths = {doc_id: sum(tf.values()) for doc_id, tf in
tf_dict.items()}
```

```

avgdl = sum(doc_lengths.values()) / N

df = Counter()
for tokens in docs_dict.values():
    for term in set(tokens):
        df[term] += 1

bm25_dict = {}
for doc_id, tf in tf_dict.items():
    vec = {}
    doc_length = doc_lengths[doc_id]
    for term, count in tf.items():
        idf = math.log((N - df[term] + 0.5) / (df[term] + 0.5) + 1)
        tf_norm = (count * (k1 + 1)) / (count + k1 * (1 - b + b *
doc_length / avgdl))
        vec[term] = idf * tf_norm
    bm25_dict[doc_id] = vec

return bm25_dict

```

Results: MAP = 0.2812 (+19.4%), P@10 = 0.296 (-11.9%)

BM25 improves MAP dramatically but hurt the P@10 slightly. That is expected, arising from the BM25 trade-off. BM25 produces better overall ranking by distributing relevant documents more effectively throughout the ranking. This increases average precision across all positions. However, this can slightly reorder the highest-ranked documents, causing a few relevant items to fall out of the top 10. As a result, early precision (P@10) may decrease even though the overall ranking quality (MAP) improves.

3.1.10 Step-9: Pseudo Relevance Feedback (PRF)

Expanded queries using top terms from top-10 retrieved documents.

```

def pseudo_relevance_feedback(query_vec, bm25_dict, top_k=10, top_terms=20):
    # first-pass ranking
    initial_results = rank_documents_for_query(query_vec, bm25_dict)
    top_docs = [docno for docno, _ in initial_results[:top_k]]

    # collect term weights from top docs

```

```

term_scores = Counter()
for docno in top_docs:
    for term, weight in bm25_dict[docno].items():
        term_scores[term] += weight

# add top new terms to query
expansion_terms = [
    t for t, _ in term_scores.most_common(top_terms + len(query_vec))
    if t not in query_vec
][:top_terms]

expanded = dict(query_vec)
for term in expansion_terms:
    expanded[term] = expanded.get(term, 0) + 0.3
return expanded

```

Results: MAP = 0.3042 (+8.2%), P@10 = 0.328 (+10.8%)

PRF provides the final improvement. By expanding the queries with terms from initially retrieved documents, we capture query relevant vocabulary that isn't present in the original short title queries. That is especially helpful for minimal queries.

3.1.11 Final English run-1 (constrained system) MAP: 0.3042, P@10: 0.328

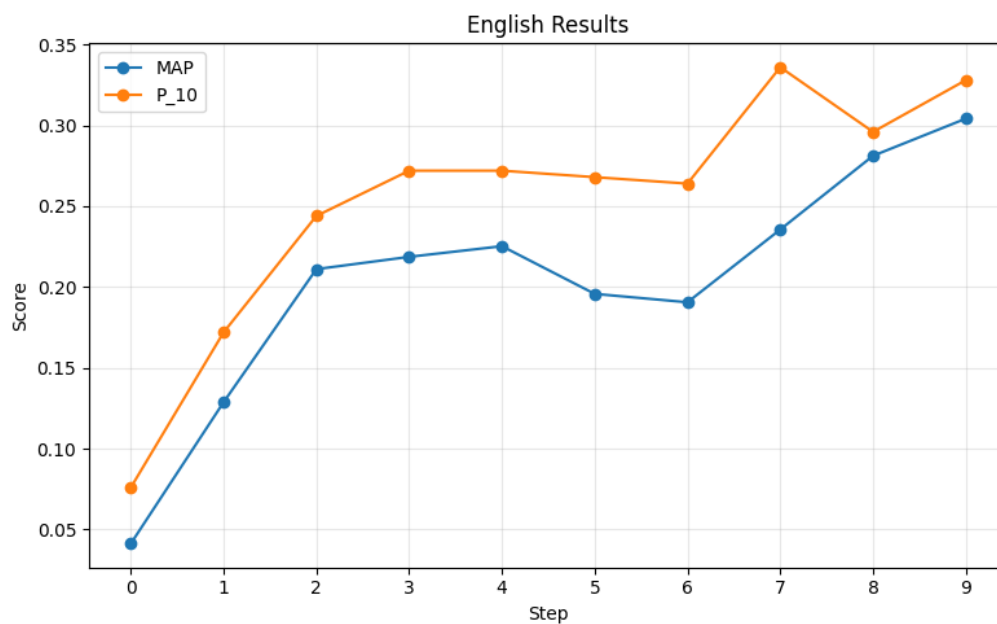


Figure 1: Performance of an English information retrieval system across iterations

3.2 Czech Experiments:

The Czech experiments followed a similar setup, but with some differences due to Czech's richer morphology.

3.2.1 Step-0: Baseline (run-0)

Same configuration as for English.

Results: MAP = 0.050, P@10 = 0.072

Similarly to English, the baseline performs poorly.

3.2.2 Step-1: Improved tokenization

Step	MAP	P@10	MAP Change
Step 1: tokenization	0.0734	0.100	+ 46.8%
Step 2: stop words removal	0.0968	0.124	+ 31.9%
Step 3: empty removal	0.0972	0.124	+ 0.4%

The pattern is resembling the English results, with the tokenization and stop words removal providing the major gains.

3.2.3 Step-4: Lemmatization

For Czech, I have used the MorphoDiTa lemmatizer.

```
def lemmatize_cs(docs_dict):  
    tagger = Tagger.load(MORPHODITA_MODEL_PATH)  
    tokenizer = tagger.newTokenizer()  
    forms = Forms()  
    lemmas = TaggedLemmas()  
    tokens = TokenRanges()  
  
    for docno, token_list in docs_dict.items():  
        text = " ".join(token_list)  
        tokenizer.setText(text)  
        doc_lemmas = []  
        while tokenizer.nextSentence(forms, tokens):  
            tagger.tag(forms, lemmas)  
            for i in range(len(lemmas)):  
                raw_lemma = lemmas[i].lemma  
                clean = raw_lemma.split("_")[0].split("-")  
                clean[0].casefold().strip()
```



```

        if clean and re.search(r"\w", clean) and clean not in
stopword_cs:

            doc_lemmas.append(clean)

    result[docno] = doc_lemmas

    return result

```

Results: MAP = 0.129 (+32.7%), P@10 = 0.148 (+19.4%)

This is the largest single improvement for Czech. Czech, unlike English, has a highly inflected morphology, consisting of 7 cases, 3 genders, and singular/plural. Lemmatization unifies all morphological variants to their lemmas, drastically increasing recall. As we discussed in class, lemmatization is more important for highly inflected languages.

3.2.4 Step-5: Equivalence classes

Similarly, as was implemented for English, but using a custom map specifically for Czech.

```

map_cs = {
    "čr": "česko", "česká republika": "česko",
    "slovenská republika": "slovensko",
    "usa": "amerika", "spojené státy": "amerika",
    "eu": "evropa", "evropská unie": "evropa",
    # ... number normalization: \d+([\.,]\d+)? → <NUM>
}

```

Results: MAP = 0.1207 (-6.4%), P@10 = 0.156 (+5.4%)

Equivalence classes hurt MAP but improved P@10 slightly. This trade-off suggests that while normalization helps precision at top ranks, it introduces noise that harms the overall AP.

3.2.5 Step-6: TF-IDF Weighting

Results: MAP = 0.1669 (+38.3%), P@10 = 0.184 (+17.9%)

Similarly to English, this step improved both MAP and P@10 dramatically.

3.2.6 Step-7: Best Match 25 (BM-25)

Results: MAP = 0.1303 (-21.9%), P@10 = 0.144 (-21.7%)

Surprisingly, BM25 underperformed TF-IDF for Czech. This may have happened due to the BM25's parameters, which may be not optimal for Czech. The smoothing factor used in the IDF (+0.5) could also be the reason we see these poor results. Overall, this result in comparison to English, highlights that optimal parameters in BM25 are language dependent, and require tuning accordingly.

3.2.7 Step-8: Pseudo Relevance Feedback (PRF)

Results: MAP = 0.1688 (+29.6%), P@10 = 0.180 (+25%)

Using PRF helped recover much of the loss from BM25 on its own, reaching the best Czech MAP result. However, these results are very close to TF-IDF. A further experiment could be tuning BM25 using testing different parameters and its effects on Czech IR systems.

3.2.8 Final Czech run-1 (constrained system) MAP: 0.1688, P@10: 0.180



Figure 2: Performance of an Czech information retrieval system across iterations

4 Final Results, Comparisons and Conclusions

System	English MAP	Czech MAP	English P@10	Czech P@10
Baseline (run-0)	0.0413	0.0500	0.076	0.072
Final (run-1)	0.3042	0.1688	0.328	0.180
Improvement	+636%	+238%	+332%	+150%

4.1 Interpolated Precision-Recall results (run-0)

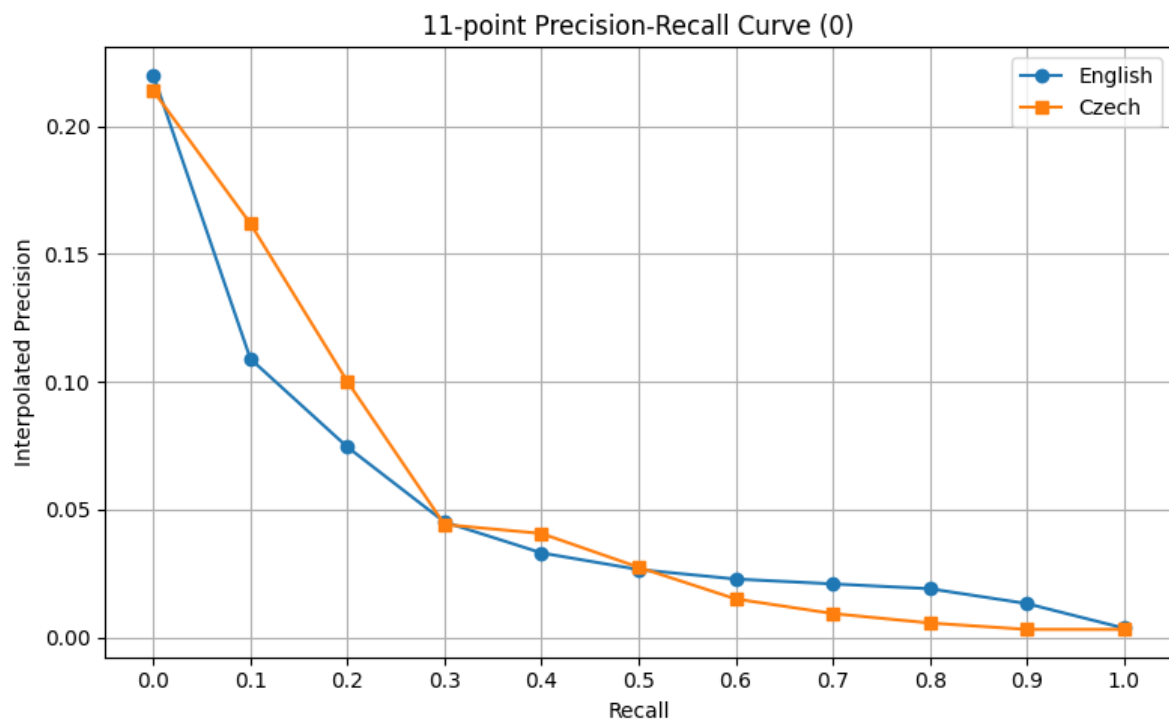


Figure 3: 11-point average precision-recall curves for the baseline system (run-0)

4.2 Interpolated Precision-Recall results (run-1)

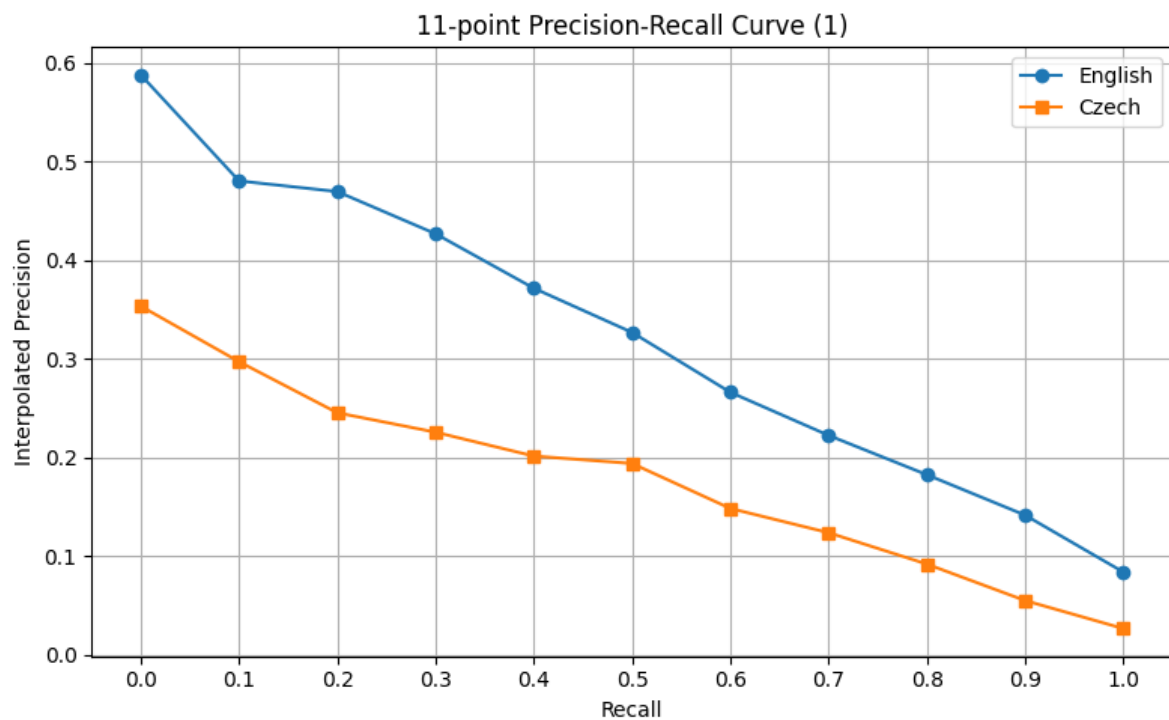


Figure 4: 11-point average precision-recall curves for the final constrained system (run-1)

Both graphs illustrate the precision-recall tradeoff. In the baseline, both curves drop to almost zero precision around a recall of 0.5, indicating that the system struggles to effectively rank relevant documents.

In run-1, there is a clear improvement for both languages, with a more pronounced gain in English. However, Czech shows a larger performance gap relative to English in the improved system than in the baseline. This may be because the BM25 model is better tuned for English than for Czech, combined with the morphological complexity of Czech, which makes retrieval more challenging.

Things that worked well:

- **Tokenization** with punctuation splitting, hyphens and parentheses was essential for both languages.
- **Stop words removal**: benefited both languages.
- **TF-IDF weighting**: provided a strong improvement for both languages.
- **Czech lemmatization**: extremely beneficial for morphologically rich languages.
- **PRF with BM25**: was effective mostly for English.

Things that didn't work:

- **English stemming**: hurt performance due to aggressive over-stemming.
- **English equivalence classes**: slightly negatively impact.
- **Czech equivalence classes**: provided a trade-off between MAP and P@10.
- **Untuned BM25 for Czech**: performed worse than TF-IDF.

The assignment highlighted the significant impact of text preprocessing and term weighting on retrieval effectiveness. It also showed that IR systems cannot be completely language independent and may require specific tuning when dealing with one language or another, as we could see with the BM25 approach which worked well for English but poorly for Czech and the lemmatization which worked poorly for English but significantly improved results for Czech.